Forcepoint Neo to Netskope Migration (Legacy Web Filtering)

Implementation Solution Guide

Version 1.0.0 · 2026-06-29

Contents

Executive Summary — Forcepoint Neo to Netskope Migration (Legacy Web Filtering) 4

What you get . . . 4

What this delivers . . . 4

The standard deployment model . . . 4

Outcome . . . 4

1. Solution Overview — Forcepoint Neo to Netskope Migration (Legacy Web Filtering) 5

What this migration is . . . 5

Why this is risky (and why a controlled migration matters) . . . 5

How it works . . . 5

Named artifacts (reused verbatim everywhere) . . . 6

What you can control . . . 6

Benefits . . . 6

Limitations (be honest with buyers) . . . 6

The standard deployment model . . . 7

2. Requirements Guide — Forcepoint Neo to Netskope Migration (Legacy Web Filtering) 7

2.1 Netskope tenant with Secure Web Gateway (SWG) . . . 7

2.2 Existing Forcepoint Neo configuration and a policy export . . . 7

2.3 Forcepoint Neo agent inventory across the estate . . . 7

2.4 Netskope client deployable via Intune . . . 8

2.5 Coexistence plan (one enforcing agent at a time) . . . 8

2.6 Migration rings (staged device groups) . . . 8

2.7 Parity test set (categories and URLs) . . . 8

2.8 Change control and a rollback path per stage . . . 8

Checklist . . . 9

3. Implementation Guide — Forcepoint Neo to Netskope Migration (Legacy Web Filtering) 9

Step 1 — Assess: inventory the legacy estate . . . 9

Step 2 — Assess: export and capture the legacy policy . . . 9

Step 3 — Configure: translate the policy into Netskope . . . 9

Step 4 — Configure: build the migration rings and groups . . . 10

Step 5 — Configure: deploy Netskope present-but-passive (coexistence) . . . 10

Step 6 — Pilot: cut over the pilot ring to Netskope . . . 10

Step 7 — Validate: parity gate on the pilot ring . . . 10

Step 8 — Validate: remove the legacy agent from the pilot ring . . . 11

Step 9 — Expand: repeat per ring . . . 11

Step 10 — Decommission Forcepoint (final gate) . . . 11

5. Testing Guide — Forcepoint Neo to Netskope Migration (Legacy Web Filtering) 11
Test 1 — Coexistence is safe (no dual enforcement) . . . 12
Test 2 — Cutover swapped enforcement . . . 12
Test 3 — Allowed traffic still works (parity) . . . 12
Test 4 — Blocked categories still blocked (parity) . . . 12
Test 5 — SSL inspection and exceptions match . . . 12
Test 6 — Legacy agent removed cleanly . . . 12
Test 7 — Coverage and conflict report . . . 12
Pass/fail gate . . . 12
6. Troubleshooting Guide — Forcepoint Neo to Netskope Migration (Legacy Web Filtering) 13
A. Both agents fighting traffic (dual enforcement) . . . 13
B. Cutover didn't take effect . . . 13
C. Parity failures (over- or under-blocking) . . . 14
D. Legacy agent won't remove . . . 14
E. Discovery / report shows unexpected devices . . . 14
F. Rollback (any ring, before legacy removal) . . . 14
G. When to escalate . . . 14
7. Best Practices — Forcepoint Neo to Netskope Migration (Legacy Web Filtering) 15
8. Deliverables Summary — Forcepoint Neo to Netskope Migration (Legacy Web Filtering) 15
Documentation . . . 15
PowerShell scripts . . . 16
Named artifacts (reused verbatim) . . . 16
What the buyer receives . . . 16
Outcome . . . 17
Appendix A — PowerShell Scripts 18

Executive Summary — Forcepoint Neo to Netskope Migration (Legacy Web Filtering)

Replacing a legacy web-filtering platform is one of the riskiest changes a network team makes. Leave the old agent and the new agent both running and they fight over the same outbound traffic; cut over too fast and you open a gap in protection or break access for users. This package gives you a controlled, repeatable migration from Forcepoint Neo to Netskope: translate the existing web-filtering policy, manage agent coexistence so the two clients do not collide, steer traffic in stages, validate parity, then decommission the legacy platform once Netskope is proven.

This is a complete, deploy-it-the-repeatable-way package, written for an administrator with no prior migration experience. Netskope and Forcepoint console internals are deferred to each vendor's official documentation; the work here is authored at the architectural and procedural level and driven from endpoint agent state and exported policy.

What you get

- A plain-language Solution Overview of the migration model, policy translation, agent coexistence and steering conflicts, and the staged cutover and decommission.
- A Requirements Guide with Description, Why, Verify, and Implement-if-missing for every prerequisite.
- An Implementation Guide of precise, ordered steps: inventory and translate legacy policy, stand up Netskope policy, steer traffic in stages, validate parity, decommission Forcepoint, then expand.
- Five PowerShell scripts for legacy-agent and policy discovery, post-migration parity validation, a migration coverage and conflict report, safe remediation (remove the legacy agent), and a migration-ring group builder.
- Testing and Troubleshooting guides with explicit pass/fail gates, plus field-proven Best Practices.

What this delivers

A staged migration with named, reusable artifacts — a translated policy set, migration rings, and device groups — moving each ring from `Forcepoint-steered` to `Netskope-steered` only after parity passes, with the legacy agent removed per ring and the platform decommissioned only after a final pilot/parity gate. A documented rollback restores the prior steering at every stage.

The standard deployment model

```
Assess -> Configure -> Pilot -> Validate -> Expand.
```

Outcome

Web filtering moved off Forcepoint Neo to Netskope with no gap in protection — policy translated, agents coexisting then cut over in stages, parity validated, and the legacy platform decommissioned — proven on a pilot before you scale.

1. Solution Overview — Forcepoint Neo to Netskope Migration (Legacy Web Filtering)

What this migration is

This is a controlled migration of web-filtering enforcement from a legacy agent-based platform (Forcepoint Neo) to Netskope's Secure Web Gateway (SWG). The legacy Forcepoint Neo agent steers and filters a device's web traffic today; the goal is to move that enforcement to the Netskope client without a gap in protection and without two agents fighting over the same outbound traffic. The work is done in stages — translate policy, run the agents side by side under controlled steering, cut over ring by ring, validate parity, then remove the legacy agent and decommission the platform.

Why this is risky (and why a controlled migration matters)

Web filtering sits inline on live user traffic. Two specific failure modes make a naive swap dangerous, and they are the reason this solution exists:

- Two agents, one traffic path. Both Forcepoint Neo and the Netskope client want to intercept and steer outbound web traffic. Run both in enforce mode at once and they collide — duplicated SSL inspection, broken certificate chains, dropped or looping connections, and unpredictable which agent actually filters a given request. Coexistence must be deliberate: one agent enforces while the other is present-but-passive, then they swap.
- Cut over too fast and you lose coverage or break access. Remove Forcepoint before Netskope policy is proven and you create a protection gap; flip Netskope to full enforce before policy parity is validated and you over-block (broken business apps) or under-block (categories that used to be denied now allowed). Staged steering with a parity gate is what prevents both.

Because of this, the migration is never a single switch. It is an assess-translate-coexist-validate sequence with a pilot/parity gate before any legacy decommission, and a documented rollback at every stage.

How it works

1. Inventory the legacy estate: which devices run the Forcepoint Neo agent, and export the current Forcepoint web-filtering policy (categories, allow/deny lists, SSL-inspection scope, exceptions).
1. Translate the legacy policy into an equivalent Netskope SWG policy set — the named `Translated Web Filtering Policy - Org - Netskope` artifact — mapping legacy categories and rules to Netskope real-time-protection policies.
1. Build migration rings (device groups) so the cutover is staged, not fleet-wide.
2. Manage coexistence: deploy the Netskope client in a present-but-passive posture alongside Forcepoint, so only one agent enforces at a time and the two never both inspect the same traffic.
1. Cut over a ring: switch that ring's steering from `Forcepoint-steered` to `Netskope-steered`, validate parity, then remove the Forcepoint Neo agent from those devices.

1. Validate parity per ring against a fixed category/URL test set, then expand to the next ring.
2. After all rings pass and a final parity gate, decommission the Forcepoint platform.

Named artifacts (reused verbatim everywhere)

Every object is named `<Purpose> - <Scope> - <Stage>` so the same string is reused across the docs and scripts:

- `Translated Web Filtering Policy - Org - Netskope` — the Netskope SWG policy set that mirrors the legacy Forcepoint web-filtering policy.
- `Migration Ring - Pilot - Cutover` — the first, small device group cut over to Netskope.
- `Migration Ring - Broad - Cutover` — the remaining fleet, cut over after the pilot passes.
- `Legacy Estate - Forcepoint Neo - Inventory` — the device group representing all endpoints still running the Forcepoint Neo agent.
- `Steering Group - Netskope - Enforce` — devices whose traffic is actively steered to and enforced by Netskope.
- `Coexistence Group - Netskope - Passive` — devices with the Netskope client installed but not yet enforcing (Forcepoint still steers).

What you can control

- Which devices are in each migration ring and the order they cut over.
- The coexistence posture per ring (which agent enforces; which is passive).
- The translated policy parity (categories, allow/deny, SSL-inspection scope, exceptions).
- The steering cutover point per ring and the legacy-agent removal per ring.
- The decommission gate — legacy platform removed only after all rings pass parity.

Benefits

- No protection gap — Netskope is proven on a ring before Forcepoint is removed from it.
- No agent collision — coexistence is deliberate; only one agent enforces a given device's traffic.
- Parity, not guesswork — a fixed test set proves the translated policy matches the legacy behavior.
- Staged blast radius — a bad translation or steering issue is caught on the pilot ring, not the fleet.
- Clean decommission — the legacy agent is removed per ring and the platform retired on evidence.

Limitations (be honest with buyers)

- Exact Netskope and Forcepoint console screens, menu paths, and policy-export steps change over time and are deferred to each vendor's official documentation and your tenant/console.
- Policy translation is a mapping exercise: legacy categories and custom rules do not always map one-to-one to Netskope categories; some require a judgment call and a parity test.

• This solution covers web-filtering enforcement migration; other Forcepoint modules (DLP, CASB, ZTNA) are separate migrations with their own parity work.

• Endpoint agent install/removal and group targeting are managed via Microsoft Intune here; non-Intune-managed estates need an equivalent deployment mechanism.

The standard deployment model

> *Inventory and translate legacy policy -> stand up the translated Netskope policy and coexistence -> cut over and validate the pilot ring -> expand ring by ring -> decommission Forcepoint after a final parity gate. Pattern:* `Assess -> Configure -> Pilot -> Validate -> Expand.`

2. Requirements Guide — Forcepoint Neo to Netskope Migration (Legacy Web Filtering)

Verify every requirement before configuring. Each entry: Description, Why, Verify, Implement-if-missing. Category-wide prerequisites (Netskope tenant, Entra ID, Intune, admin roles, network fundamentals, pilot group, change control) are in `global-requirements-network.md` and are assumed in addition to the migration-specific items below.

> *Vendor note: exact Netskope and Forcepoint console screens, menu paths, and policy-export and client-deployment steps change over time and are deferred to each vendor's official documentation and your tenant/console. Follow the named step and confirm the precise control there.*

2.1 Netskope tenant with Secure Web Gateway (SWG)

Description: An active Netskope tenant with the SWG / real-time web-protection capability licensed. Why: SWG is the destination control plane that will enforce the translated web-filtering policy. Verify: Sign in to the Netskope admin console and confirm SWG / real-time protection policy is available (deferred to Netskope docs for the exact location). Implement: Engage your Netskope account team / reseller to enable SWG on the tenant.

2.2 Existing Forcepoint Neo configuration and a policy export

Description: Admin access to the Forcepoint Neo console and an export of the current web-filtering policy (categories, allow/deny lists, SSL-inspection scope, exceptions). Why: The export is the source of truth for what the translated Netskope policy must match for parity. Verify: Open the Forcepoint Neo console and produce a policy export (deferred to Forcepoint docs for the exact export path); run `NSMIG-Discovery.ps1` to confirm which devices run the agent. Implement: Obtain Forcepoint console admin access; export the active web-filtering policy and record it as the parity baseline.

2.3 Forcepoint Neo agent inventory across the estate

Description: A known, current list of which managed devices still run the Forcepoint Neo agent — the `Legacy Estate - Forcepoint Neo - Inventory` group. Why: You cannot stage a cutover or confirm decommission without knowing where the legacy agent is installed. Verify: Run `NSMIG-Discovery.ps1` (detects the Forcepoint Neo agent on a device); cross-check against the Intune managed-device inventory. Implement: Build the `Legacy Estate - Forcepoint Neo - Inventory`

group from discovery output; keep it current as agents are removed.

2.4 Netskope client deployable via Intune

Description: The Netskope client (and any required root/decryption certificates and steering config) packaged for deployment to managed devices through Microsoft Intune. Why: The new agent and its certificates must reach endpoints to enforce SSL inspection and steering. Verify: Confirm the Netskope client app/config and certificate profiles exist in Intune and can target a group; install paths are deferred to Netskope deployment docs. Implement: Package the Netskope client per Netskope's deployment guidance; create the Intune app, certificate, and steering-config profiles.

2.5 Coexistence plan (one enforcing agent at a time)

Description: A defined posture where the Netskope client is installed present-but-passive while Forcepoint still enforces, with a single, controlled point where enforcement swaps. Why: Two agents enforcing the same traffic collide (duplicated SSL inspection, broken certificate chains, dropped/looping connections). Verify: Confirm the passive-vs-enforce posture per ring is documented and that `Coexistence Group - Netskope - Passive` and `Steering Group - Netskope - Enforce` are distinct, non-overlapping groups. Implement: Define the coexistence posture with Netskope's guidance (deploy passive, then enforce); never enroll a device in both the passive and enforce groups at once.

2.6 Migration rings (staged device groups)

Description: Entra security groups `Migration Ring - Pilot - Cutover` (small, representative) and `Migration Ring - Broad - Cutover` (the rest), plus the steering/coexistence groups above. Why: Cutover must be staged so a bad translation or steering issue is caught on the pilot, not the fleet. Verify: Entra -> Groups; the pilot and broad migration rings exist and membership is non-overlapping; run `NSMIG-Set-MigrationRingTargeting.ps1 -WhatIf` to preview. Implement: Create the groups with `NSMIG-Set-MigrationRingTargeting.ps1` (or Entra -> Groups -> New group -> Security) and populate the pilot ring first.

2.7 Parity test set (categories and URLs)

Description: A fixed list of representative URLs per legacy category (allowed, blocked, SSL-inspected, exception) used to compare Forcepoint behavior to Netskope behavior. Why: Parity is proven by reproducible tests, not by assumption that the translation is complete. Verify: The test set covers each category and rule in the policy export; results are recorded per ring. Implement: Derive the test set from the `Translated Web Filtering Policy - Org - Netskope` mapping and the legacy export; keep it as the regression baseline.

2.8 Change control and a rollback path per stage

Description: A change window and a documented rollback (revert steering to `Forcepoint-steered`, set Netskope back to passive, do not remove the legacy agent) for each ring. Why: Inline web-filtering changes affect live traffic and every cutover stage must be reversible until parity passes. Verify: A per-ring rollback is written and approved before that ring is steered to Netskope. Implement: Define rollback as revert-steering + Netskope-passive; only remove the Forcepoint agent after the ring passes parity and the change is approved.

Checklist

- Netskope tenant with SWG licensed
- Forcepoint Neo console access + web-filtering policy export (parity baseline)
- Forcepoint Neo agent inventory — `NSMIG-Discovery.ps1`
- Netskope client + certificates deployable via Intune
- Coexistence plan: one enforcing agent at a time (passive vs enforce groups)
- Migration rings created — `NSMIG-Set-MigrationRingTargeting.ps1 -WhatIf`
- Parity test set (categories/URLs) defined
- Per-ring change control + rollback approved

3. Implementation Guide — Forcepoint Neo to Netskope Migration (Legacy Web Filtering)

Follow these steps in order. Endpoint agent and group work is done in the Microsoft Intune admin center (`https://intune.microsoft.com`) and Microsoft Entra (`https://entra.microsoft.com`). Netskope and Forcepoint console steps are deferred to each vendor's official documentation — the named action is given; confirm the exact control in your console.

> *Model: translate the legacy policy, deploy Netskope present-but-passive alongside Forcepoint, cut over one ring at a time, validate parity, remove the legacy agent per ring, and decommission the platform only after a final parity gate. Pattern:* `Assess -> Configure -> Pilot -> Validate -> Expand.`

Step 1 — Assess: inventory the legacy estate

1. Run `NSMIG-Discovery.ps1` on representative devices (or via your management tooling) to detect the Forcepoint Neo agent and capture its version and the device's current web-steering state.

2. In Entra -> Groups -> New group -> Security, create `Legacy Estate - Forcepoint Neo - Inventory` and populate it from the discovery output. This group tracks where the legacy agent still lives so you can confirm decommission later.

Step 2 — Assess: export and capture the legacy policy

1. In the Forcepoint Neo console, export the active web-filtering policy — categories, allow/deny lists, SSL-inspection scope, and exceptions (exact export path deferred to Forcepoint docs).

2. Record the export as the parity baseline. Derive the parity test set (Requirement 2.7): representative URLs for each allowed, blocked, SSL-inspected, and exception rule.

Step 3 — Configure: translate the policy into Netskope

1. In the Netskope console, build the SWG real-time-protection policy set named `Translated Web Filtering Policy - Org - Netskope` that mirrors the legacy export (exact policy screens deferred to Netskope docs).

2. Map each legacy category and custom rule to its Netskope equivalent. Where a legacy category has no one-to-one Netskope category, choose the closest mapping and flag it for a parity test.

3. Do not assign this policy to enforce yet — at this stage Netskope is configured but not steering live traffic for any production device.

Step 4 — Configure: build the migration rings and groups

1. Run `NSMIG-Set-MigrationRingTargeting.ps1 -WhatIf` to preview, then run it to create/populate the Entra groups: `Migration Ring - Pilot - Cutover` (small, representative), `Migration Ring - Broad - Cutover` (the rest), `Coexistence Group - Netskope - Passive`, and `Steering Group - Netskope - Enforce`.

2. Keep membership non-overlapping: a device is in either the passive group or the enforce group, never both.

Step 5 — Configure: deploy Netskope present-but-passive (coexistence)

1. In Intune, assign the Netskope client app, certificate profiles, and steering config to `Coexistence Group - Netskope - Passive` (which initially contains `Migration Ring - Pilot - Cutover`).

2. Deploy Netskope in a passive posture so Forcepoint continues to steer and enforce; Netskope is installed but not inspecting the same traffic (passive-vs-enforce posture deferred to Netskope docs).

3. On a pilot device, run `NSMIG-Validation.ps1` to confirm both agents are present and that only Forcepoint is currently steering (no dual enforcement).

Step 6 — Pilot: cut over the pilot ring to Netskope

1. Move the pilot ring out of passive and into enforce: remove `Migration Ring - Pilot - Cutover` devices from `Coexistence Group - Netskope - Passive` and add them to `Steering Group - Netskope - Enforce` (use `NSMIG-Set-MigrationRingTargeting.ps1`).

2. In Netskope, ensure `Translated Web Filtering Policy - Org - Netskope` enforces for the enforce group, and set Forcepoint to stop steering those devices (exact steering controls deferred to each vendor's docs).

3. Confirm the swap: exactly one agent now enforces. Run `NSMIG-Validation.ps1` — it should report Netskope steering active and Forcepoint no longer steering on pilot devices.

Step 7 — Validate: parity gate on the pilot ring

1. Run the parity test set on pilot devices: each allowed URL still loads, each blocked category is still blocked, SSL-inspected sites present the expected certificate, and exceptions behave as before.

2. Run `NSMIG-Report.ps1` for a coverage and conflict report (devices steered, agents present, parity status). Complete the Testing Guide.

3. Parity gate: do not proceed until the pilot ring passes. If it fails, roll back (Step 6 reversed: return the ring to passive, revert steering to Forcepoint, do not remove the legacy agent) and fix the translation.

Step 8 — Validate: remove the legacy agent from the pilot ring

1. Only after the pilot passes parity, remove the Forcepoint Neo agent from pilot devices. Run `NSMIG-Remediation.ps1` (default `-WhatIf`) to preview, then re-run with `-Confirm` to remove the legacy agent.

2. Run `NSMIG-Validation.ps1` to confirm Forcepoint Neo is gone and Netskope is the sole enforcing agent. Update `Legacy Estate - Forcepoint Neo - Inventory` to drop the removed devices.

Step 9 — Expand: repeat per ring

1. Add `Migration Ring - Broad - Cutover` devices to `Coexistence Group - Netskope - Passive`, deploy Netskope passive (Step 5), then repeat Steps 6-8 for the broad ring.

2. Re-run `NSMIG-Report.ps1` each ring to confirm coverage grows and no device is left dual-enforced or unprotected.

Step 10 — Decommission Forcepoint (final gate)

1. Final parity gate: confirm `Legacy Estate - Forcepoint Neo - Inventory` is empty (every device has had the agent removed and validated) and `NSMIG-Report.ps1` shows full Netskope coverage with no conflicts.

2. Only then decommission the Forcepoint Neo platform (retire the tenant/console and management infrastructure per Forcepoint's guidance).

Order of operations

```
Inventory -> policy export -> translate -> rings/groups -> Netskope passive -> pilot cutover -> pilot parity gate -> remove legacy agent (pilot) -> repeat per ring -> decommission Forcepoint
```

Rollback note

At any ring before the legacy agent is removed, roll back by returning the ring to `Coexistence Group - Netskope - Passive`, reverting steering to `Forcepoint-steered`, and leaving the Forcepoint Neo agent installed. Legacy-agent removal (Step 8) is the point of no easy return — it is gated on a passed parity test for that ring.

5. Testing Guide — Forcepoint Neo to Netskope Migration (Legacy Web Filtering)

Run per ring after that ring is cut over (Implementation Steps 6-8), and re-run the parity tests whenever the translated policy changes. The pilot ring must pass before any broad ring; the legacy agent is removed from a ring only after that ring passes.

Test 1 — Coexistence is safe (no dual enforcement)

Do: On a passive-stage device run `NSMIG-Validation.ps1`. Expected: Both agents may be present, but only one is steering — Forcepoint enforces while Netskope is passive. No dual SSL inspection.

Test 2 — Cutover swapped enforcement

Do: After moving a ring to `Steering Group - Netskope - Enforce`, run `NSMIG-Validation.ps1` on those devices. Expected: Netskope is steering/enforcing; Forcepoint is no longer steering. Exactly one enforcing agent.

Test 3 — Allowed traffic still works (parity)

Do: From a cut-over device, browse the allowed URLs in the parity test set. Expected: Each allowed site loads as it did under Forcepoint — no new over-blocking.

Test 4 — Blocked categories still blocked (parity)

Do: Browse the blocked-category URLs in the parity test set. Expected: Each is blocked by Netskope, matching the legacy policy — no under-blocking.

Test 5 — SSL inspection and exceptions match

Do: Visit SSL-inspected sites and exception URLs from the test set. Expected: Inspected sites present the expected Netskope certificate; exceptions bypass inspection exactly as the legacy policy did.

Test 6 — Legacy agent removed cleanly

Do: After `NSMIG-Remediation.ps1` runs for a ring, run `NSMIG-Validation.ps1`. Expected: Forcepoint Neo is no longer present; Netskope is the sole enforcing agent; the device still filters correctly.

Test 7 — Coverage and conflict report

Do: Run `NSMIG-Report.ps1`. Expected: Per-device view of agents present and steering owner; no device is dual-enforced or left unprotected; the `Legacy Estate - Forcepoint Neo - Inventory` count shrinks as rings complete.

Pass/fail gate

Gate	Requirement
Coexistence safe	Test 1 passes for the ring (no dual enforcement) before cutover
Pilot parity gate	Tests 2-5 pass on `Migration Ring - Pilot - Cutover` before the legacy agent is removed or any broad ring starts

Gate	Requirement
Per-ring complete	Tests 2-6 pass for the ring; report (Test 7) shows it covered with no conflict
Decommission gate	Test 7 shows `Legacy Estate - Forcepoint Neo - Inventory` empty and full Netskope coverage before Forcepoint is decommissioned

6. Troubleshooting Guide — Forcepoint Neo to Netskope Migration (Legacy Web Filtering)

Use `NSMIG-Validation.ps1` / `NSMIG-Discovery.ps1` for device agent state, `NSMIG-Report.ps1` for fleet coverage and conflicts, and `NSMIG-Remediation.ps1` for safe legacy-agent removal. Netskope and Forcepoint console specifics are deferred to each vendor's documentation.

A. Both agents fighting traffic (dual enforcement)

Symptom	Cause	Remediation
Broken HTTPS, certificate errors, looping/dropped connections	Forcepoint and Netskope both enforcing the same traffic	Run `NSMIG-Validation.ps1`; ensure the device is in only one of `Coexistence Group - Netskope - Passive` or `Steering Group - Netskope - Enforce`; set Netskope back to passive until the swap is intended
Random which site is filtered by which agent	Overlapping group membership	Remove the overlap; a device must be passive or enforce, never both

B. Cutover didn't take effect

Symptom	Cause	Remediation
Ring moved to enforce but Forcepoint still steering	Steering not switched in the consoles, or policy not assigned	Confirm `Translated Web Filtering Policy - Org - Netskope` is assigned to the enforce group and Forcepoint steering is stopped for those devices (vendor consoles); re-sync Intune; re-run `NSMIG-Validation.ps1`
Netskope not steering after enforce	Client/cert not deployed or device not synced	Confirm the Netskope client and certificate profiles are assigned and installed; sync the device

C. Parity failures (over- or under-blocking)

Symptom	Cause	Remediation
Sites that worked now blocked	Translated policy too strict or a mis-mapped category	Compare against the policy export; adjust the Netskope category/allow rule; re-test that case
Previously blocked categories now allowed	Missing/mis-mapped rule in the translation	Add the equivalent Netskope rule; re-run the blocked-category parity tests
SSL inspection breaks an app	Inspection scope or exception not carried over	Recreate the legacy exception / inspection scope in Netskope; re-test

D. Legacy agent won't remove

- Run `NSMIG-Remediation.ps1` first with default `-WhatIf` to confirm what it would do, then with `-Confirm`. If removal fails, the Forcepoint Neo agent may have tamper protection or require an uninstall password — obtain it from the Forcepoint console/admin (deferred to Forcepoint docs) before retrying.
- Do not remove the legacy agent until the ring has passed the parity gate; premature removal opens a protection gap.

E. Discovery / report shows unexpected devices

- A device still appears in `Legacy Estate - Forcepoint Neo - Inventory` after removal: re-run `NSMIG-Discovery.ps1` to refresh state, then update the group; the decommission gate requires this group to be empty.
- New devices appear with the legacy agent: add them to the appropriate migration ring and run the standard sequence; do not decommission the platform while any device still reports the agent.

F. Rollback (any ring, before legacy removal)

- Return the ring to `Coexistence Group - Netskope - Passive`, revert steering to Forcepoint, and leave the Forcepoint Neo agent installed. This restores the pre-cutover state with no protection gap. Re-run `NSMIG-Validation.ps1` to confirm Forcepoint is steering again.

G. When to escalate

If coexistence is correct (one enforcing agent), the translated policy matches the export, and parity tests still fail or the cutover will not swap, capture `NSMIG-Validation.ps1` and `NSMIG-Report.ps1` output and open a support case with the relevant vendor (Netskope for steering/policy, Forcepoint for agent removal/steering teardown).

7. Best Practices — Forcepoint Neo to Netskope Migration (Legacy Web Filtering)

• One enforcing agent at a time. Never run Forcepoint Neo and Netskope both enforcing the same traffic — deploy Netskope passive, then swap. Dual SSL inspection is the fastest way to break access fleet-wide.

• Translate from an export, not from memory. Use the exported Forcepoint policy as the parity baseline; map every category, rule, and exception, and flag any mapping that is not one-to-one for a parity test.

• Migrate in rings. A small `Migration Ring - Pilot - Cutover` catches a bad translation or steering issue before the `Migration Ring - Broad - Cutover` fleet — the single most important practice.

• Gate decommission on parity, not on schedule. Remove the Forcepoint Neo agent from a ring only after that ring passes the parity test set; decommission the platform only after `Legacy Estate - Forcepoint Neo - Inventory` is empty.

• Keep groups non-overlapping. A device is in either `Coexistence Group - Netskope - Passive` or `Steering Group - Netskope - Enforce`, never both — overlap causes the dual-enforcement collision.

• Preview before you remove. `NSMIG-Remediation.ps1` defaults to `-WhatIf`; review the planned removals before re-running with `-Confirm`.

• Test allowed and blocked, not just blocked. Over-blocking breaks the business as surely as under-blocking exposes it — the parity set must include known-good URLs.

• Keep a rollback ready per ring. Until the legacy agent is removed, rollback is cheap: return the ring to passive and revert steering to Forcepoint. Plan it before each cutover.

• Reuse the named artifacts verbatim. The same `<Purpose> - <Scope> - <Stage>` names in the docs, scripts, and consoles keep the migration auditable and repeatable.

• Defer console specifics to the vendors. Netskope and Forcepoint screens change; operate on agent state and policy export at the architectural level and confirm the exact control in each vendor's current docs.

• Document the cadence. Record which ring cut over when, its parity result, and when its legacy agent was removed — for audit and for the decommission gate.

8. Deliverables Summary — Forcepoint Neo to Netskope Migration (Legacy Web Filtering)

Documentation

Document	Purpose
Executive Summary	Business case and what's in the box

Document	Purpose
1. Solution Overview	The migration model, policy translation, agent coexistence and steering conflicts, staged cutover and decommission, and why it matters
2. Requirements Guide	Every prerequisite with Verify / Implement-if-missing
Global Requirements (Network)	Category-wide prerequisites, published once
3. Implementation Guide	Exact, ordered steps: inventory, translate, coexist, cut over per ring, validate parity, remove legacy agent, decommission
5. Testing Guide	Validation procedures with pass/fail and parity gates
6. Troubleshooting Guide	Field issues and remediation
7. Best Practices	Field-proven recommendations

PowerShell scripts

Script	What it does
`NSMIG-Discovery.ps1`	Detects the Forcepoint Neo agent and current web-steering state on a device (read-only)
`NSMIG-Validation.ps1`	Confirms coexistence/cutover state: which agents are present and which is steering (read-only)
`NSMIG-Report.ps1`	Fleet coverage and conflict report from Intune (agents present, steering owner, legacy estate) via Microsoft Graph (read-only)
`NSMIG-Remediation.ps1`	Safe remediation; removes the legacy Forcepoint Neo agent (defaults to `-WhatIf`)
`NSMIG-Set-MigrationRingTargeting.ps1`	Builds/populates the migration-ring, coexistence, and enforce Entra groups via Microsoft Graph

Named artifacts (reused verbatim)

- `Translated Web Filtering Policy - Org - Netskope`
- `Migration Ring - Pilot - Cutover`
- `Migration Ring - Broad - Cutover`
- `Legacy Estate - Forcepoint Neo - Inventory`
- `Steering Group - Netskope - Enforce`
- `Coexistence Group - Netskope - Passive`

What the buyer receives

- This implementation guide as a branded Word document and PDF.
- All five PowerShell scripts in a single ZIP.
- A repeatable process: `Assess -> Configure -> Pilot -> Validate -> Expand.`

Outcome

Web filtering moved off Forcepoint Neo to Netskope with no gap in protection — policy translated, agents coexisting then cut over in stages, parity validated, and the legacy platform decommissioned after a final parity gate.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

NSMIG-Discovery.ps1

Purpose: Reports whether the legacy Forcepoint Neo agent is installed on this device (service and installed-product detection), its version if found, and whether a Netskope client is also present — the inputs you need to build the Legacy Estate - Forcepoint Neo - Inventory group and plan coexistence. Netskope/Forcepoint console steering specifics are deferred to vendor docs; this script reports only locally observable agent state. HOW TO RUN Elevated: powershell -ExecutionPolicy Bypass -File .\NSMIG-Discovery.ps1 EXPECTED RESULTS Forcepoint Neo present (yes/no) + version, Netskope client present (yes/no), and a coexistence note if both are installed. Read-only; makes no changes. .NOTES Network Solutions. Read-only. Netskope/Forcepoint console specifics deferred to vendor docs. #> [CmdletBinding()] param() Write-Host "`n=== Forcepoint Neo -> Netskope migration — discovery ($($env:COMPUTERNAME)) ===" -ForegroundColor Cyan

How to run: Elevated: powershell -ExecutionPolicy Bypass -File .\NSMIG-Discovery.ps1 EXPECTED RESULTS Forcepoint Neo present (yes/no) + version, Netskope client present (yes/no), and a coexistence note if both are installed. Read-only; makes no changes. .NOTES Network Solutions. Read-only. Netskope/Forcepoint console specifics deferred to vendor docs. #> [CmdletBinding()] param() Write-Host "`n=== Forcepoint Neo -> Netskope migration — discovery ($($env:COMPUTERNAME)) ===" -ForegroundColor Cyan

NSMIG-Remediation.ps1

Purpose: Safely removes the legacy Forcepoint Neo web-filtering agent from a device AFTER that device's migration ring has passed the parity gate. By default it runs in preview (-WhatIf) and only reports what it would uninstall; pass -Confirm to actually remove. It locates the agent's uninstall string from the registry and invokes it. It does NOT remove the Netskope client. Some Forcepoint Neo builds enforce tamper protection / an uninstall password — that is deferred to Forcepoint console/admin (supply it per Forcepoint docs if removal is blocked). HOW TO RUN Preview (default): powershell -ExecutionPolicy Bypass -File .\NSMIG-Remediation.ps1 Remove for real: powershell -ExecutionPolicy Bypass -File .\NSMIG-Remediation.ps1 -Confirm EXPECTED RESULTS -WhatIf: prints the uninstall command(s) it would run, changes nothing. -Confirm: invokes the Forcepoint Neo uninstaller; re-run NSMIG-Validation.ps1 afterward. .NOTES Network Solutions. Defaults to -WhatIf. Only remove the legacy agent after the ring passes parity. #> [CmdletBinding()] param([switch]$WhatIf = $true, [switch]$Confirm) if ($Confirm) { $WhatIf = $false } Write-Host "=== NSMIG Remediation — remove Forcepoint Neo agent ($($env:COMPUTERNAME)) ===" -ForegroundColor Cyan Write-Host ("Mode: {0}" -f ($(if($WhatIf){'PREVIEW (-WhatIf) — no changes'}else{'REMOVE (-Confirm)'}))) -ForegroundColor ($(if($WhatIf){'Yellow'}else{'Red'}))

How to run: Preview (default): powershell -ExecutionPolicy Bypass -File .\NSMIG-Remediation.ps1 Remove for real: powershell -ExecutionPolicy Bypass -File .\NSMIG-Remediation.ps1 -Confirm EXPECTED RESULTS -WhatIf: prints the uninstall command(s) it would run, changes nothing. -Confirm: invokes the Forcepoint Neo uninstaller; re-run NSMIG-Validation.ps1 afterward. .NOTES Network Solutions. Defaults to -WhatIf. Only remove the legacy agent after the ring passes parity. #> [CmdletBinding()] param([switch]$WhatIf = $true, [switch]$Confirm) if ($Confirm) { $WhatIf = $false } Write-Host "=== NSMIG Remediation — remove Forcepoint Neo agent ($($env:COMPUTERNAME)) ==="

-ForegroundColor Cyan Write-Host ("Mode: {0}" -f ($(if($WhatIf){'PREVIEW (-WhatIf) — no changes'}else{'REMOVE (-Confirm)'}))) -ForegroundColor ($(if($WhatIf){'Yellow'}else{'Red'}))

NSMIG-Report.ps1

Purpose: Summarizes the migration across managed Windows devices using Intune detected-apps data: which devices still have the Forcepoint Neo agent, which have the Netskope client, and which have BOTH (a coexistence/conflict watch item). Gives you the legacy-estate count for the decommission gate and flags devices that are dual-equipped. Steering-ownership in the consoles is deferred to Netskope/Forcepoint docs; this reports installed-agent coverage from Intune. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\NSMIG-Report.ps1 -CsvPath C:\Temp\nsmig.csv Requires the Microsoft Graph PowerShell SDK and DeviceManagementManagedDevices.Read.All. EXPECTED RESULTS Counts of Forcepoint-only, Netskope-only, both, and neither; a per-device CSV; and a flag if any device has both agents. Read-only against Graph. .NOTES Network Solutions. Read-only against Graph. Netskope/Forcepoint console specifics deferred to vendor docs. #> [CmdletBinding()] param([string]$CsvPath) if (-not (Get-Module -ListAvailable -Name Microsoft.Graph.Authentication)) { Install-Module Microsoft.Graph -Scope CurrentUser -Force -AllowClobber } Import-Module Microsoft.Graph.Authentication -ErrorAction Stop Connect-MgGraph -Scopes 'DeviceManagementManagedDevices.Read.All' -NoWelcome

How to run: powershell -ExecutionPolicy Bypass -File .\NSMIG-Report.ps1 -CsvPath C:\Temp\nsmig.csv Requires the Microsoft Graph PowerShell SDK and DeviceManagementManagedDevices.Read.All. EXPECTED RESULTS Counts of Forcepoint-only, Netskope-only, both, and neither; a per-device CSV; and a flag if any device has both agents. Read-only against Graph. .NOTES Network Solutions. Read-only against Graph. Netskope/Forcepoint console specifics deferred to vendor docs. #> [CmdletBinding()] param([string]$CsvPath) if (-not (Get-Module -ListAvailable -Name Microsoft.Graph.Authentication)) { Install-Module Microsoft.Graph -Scope CurrentUser -Force -AllowClobber } Import-Module Microsoft.Graph.Authentication -ErrorAction Stop Connect-MgGraph -Scopes 'DeviceManagementManagedDevices.Read.All' -NoWelcome

NSMIG-Set-MigrationRingTargeting.ps1

Purpose: Ensures the named Entra security groups used to stage the Forcepoint Neo -> Netskope cutover exist, and (optionally) moves a device between the passive and enforce groups for a ring swap. Groups created/ensured (names reused verbatim across the docs and consoles): Migration Ring - Pilot - Cutover Migration Ring - Broad - Cutover Legacy Estate - Forcepoint Neo - Inventory Coexistence Group - Netskope - Passive Steering Group - Netskope - Enforce Defaults to -WhatIf (preview). Group/membership changes happen only with -Confirm. Netskope steering and Forcepoint teardown in the consoles remain manual and are deferred to vendor docs. HOW TO RUN Preview groups: powershell -ExecutionPolicy Bypass -File .\NSMIG-Set-MigrationRingTargeting.ps1 Create groups: powershell -ExecutionPolicy Bypass -File .\NSMIG-Set-MigrationRingTargeting.ps1 -Confirm Swap a device to enforce (cutover): .\NSMIG-Set-MigrationRingTargeting.ps1 -Confirm -DeviceObjectId -SwapToEnforce Requires the Microsoft Graph PowerShell SDK and Group.ReadWrite.All. EXPECTED RESULTS -WhatIf: lists which groups exist and which it would create / which membership it would change. -Confirm: ensures the groups exist; with -SwapToEnforce, moves the device from the passive group to the enforce group (one enforcing agent at a time). .NOTES Network Solutions. Defaults to -WhatIf. Keeps passive and enforce membership non-overlapping. #> [CmdletBinding()] param([switch]$WhatIf = $true, [switch]$Confirm, [string]$DeviceObjectId, [switch]$SwapToEnforce) if ($Confirm) { $WhatIf = $false }

How to run: Preview groups: powershell -ExecutionPolicy Bypass -File .\NSMIG-Set-MigrationRingTargeting.ps1 Create groups: powershell -ExecutionPolicy Bypass -File .\NSMIG-Set-MigrationRingTargeting.ps1 -Confirm Swap a device to enforce (cutover): .\NSMIG-Set-MigrationRingTargeting.ps1 -Confirm -DeviceObjectId <id> -SwapToEnforce Requires the Microsoft Graph PowerShell SDK and Group.ReadWrite.All. EXPECTED RESULTS -WhatIf: lists which groups exist and which it would create / which membership it would change. -Confirm: ensures the groups exist; with -SwapToEnforce, moves the device from the passive group to the enforce group (one enforcing agent at a time). .NOTES Network Solutions. Defaults to -WhatIf. Keeps passive and enforce membership non-overlapping. #> [CmdletBinding()] param([switch]$WhatIf = $true, [switch]$Confirm, [string]$DeviceObjectId, [switch]$SwapToEnforce) if ($Confirm) { $WhatIf = $false }

NSMIG-Validation.ps1

Purpose: Verifies the migration state of a device: which agents are present (Forcepoint Neo, Netskope) and whether the device is safely in one of the expected stages — passive coexistence (Forcepoint present and running, Netskope present but not enforcing) or cut over (Netskope present and running, Forcepoint removed/stopped). Flags the dangerous dual-enforcement state. Steering ownership confirmation in the consoles is deferred to Netskope/Forcepoint docs; this checks locally observable agent/service state. HOW TO RUN Elevated: powershell -ExecutionPolicy Bypass -File .\NSMIG-Validation.ps1 EXPECTED RESULTS A stage verdict — "PASSIVE COEXISTENCE", "CUT OVER TO NETSKOPE", or a warning ("DUAL ENFORCEMENT RISK" / "UNPROTECTED"). Read-only; exit 0 on a clean stage, 1 otherwise. .NOTES Network Solutions. Read-only. Netskope/Forcepoint console steering specifics deferred to vendor docs. #> [CmdletBinding()] param() Write-Host "=== NSMIG Validation — $($env:COMPUTERNAME) ===" -ForegroundColor Cyan

How to run: Elevated: powershell -ExecutionPolicy Bypass -File .\NSMIG-Validation.ps1 EXPECTED RESULTS A stage verdict — "PASSIVE COEXISTENCE", "CUT OVER TO NETSKOPE", or a warning ("DUAL ENFORCEMENT RISK" / "UNPROTECTED"). Read-only; exit 0 on a clean stage, 1 otherwise. .NOTES Network Solutions. Read-only. Netskope/Forcepoint console steering specifics deferred to vendor docs. #> [CmdletBinding()] param() Write-Host "=== NSMIG Validation — $($env:COMPUTERNAME) ===" -ForegroundColor Cyan